

15 — Greedy

DSA in Python — Interview Prep

Greedy — Tutorial

Covers the 10 problems solved in this folder. A greedy algorithm makes the locally-best choice at every step and never looks back or reconsiders — which only produces a correct *global* answer when the problem has a specific structural property (an "exchange argument": any optimal solution can be transformed into the greedy choice without making things worse). The hard part of a greedy problem is rarely writing the loop — it's proving to yourself *why* the local choice is safe, which is what most of the docstrings in this folder focus on.

The patterns

1. Prefer the more flexible resource (01)

`lemonade_change`'s only real decision point is making change for a \$20: spend a \$10 (if available) before spending three \$5s, because a \$5 bill can make change for *either* a future \$10 or \$20, while a \$10 bill only ever helps with a future \$20. Keeping the more versatile resource in reserve is never worse — a small but general greedy instinct.

2. Decompose a global optimum into local gains (02)

`best_time_to_buy_and_sell_stock_ii` capturing every single positive day-to-day price increase is provably equivalent to the true optimum, because any longer profitable buy-low-sell-high stretch is exactly the sum of its individual daily gains — there's nothing to be gained by trying to detect longer runs explicitly.

3. Track one running "farthest reach" value (03, 04)

Both jump-game problems avoid trying every possible jump length from every position by tracking only the single most useful fact: how far could be reached, at best, from everything considered so far. `can_jump` (03) just checks that value never falls behind the current position. `jump` (04) turns the same tracked value into an implicit BFS: `curr_end` marks the current "level's" boundary, and hitting it forces a jump-count increment before the level (window) advances — the same "process a whole distance-level before moving to the next" shape as a real level-order BFS, expressed without an actual queue.

4. Reset the search boundary instead of retrying every candidate (05)

`gas_station`'s key insight is that once the running tank goes negative between `start` and some index `i`, *no* station in that whole stretch could have worked as a starting point — so the next candidate start can jump straight past all of them to `i + 1`, instead of retrying each one individually. This is what keeps the whole algorithm $O(n)$ instead of $O(n^2)$.

5. Sort by end time for interval scheduling (06, 07)

Both interval problems use the same greedy backbone: sort by end coordinate, then greedily keep/act on whichever interval ends soonest, since that leaves the most room for everything that comes after.

`non_overlapping_intervals` counts what has to be *rejected* when a new interval starts before the last kept one ends; `minimum_arrows_to_burst_balloons` is the same idea from the opposite angle — placing a

"covering" arrow at the earliest interval's end, since anything that extends that far is covered for free, and only starting a new arrow once something falls outside the current one's reach.

6. Extend a boundary to cover everything currently active (08)

`partition_labels` scans once, extending the current partition's `end` to the last occurrence of every character seen so far — closing the partition only once the scan position catches up to that boundary, guaranteeing every character inside has no future occurrence left unaccounted for. Structurally the same "extend to cover what's currently reachable/active" idea as the multi-source traversals in `11_graphs`, just applied in one dimension over a string instead of a grid.

7. Two passes to satisfy a bidirectional constraint (09)

`candy`'s constraint compares each child to *both* neighbors, but a single greedy pass can only correctly enforce a comparison in one direction. The fix — a left-to-right pass, then a right-to-left pass using `max` (not `overwrite`) to avoid undoing what the first pass already guaranteed — is the general template whenever a greedy constraint needs to look both ways: split it into two one-directional passes and combine with whichever operation preserves both.

8. Find a processing order where one constraint can't undo another (10)

`queue_reconstruction_by_height` has two coordinates pulling in different directions. The trick is finding the *order* to process people in where satisfying one person's constraint never retroactively breaks an already-placed person's constraint: tallest-first (with ties broken by ascending `k`) means every already-placed person is at least as tall, so inserting a new (necessarily shorter-or-equal) person anywhere never changes any earlier person's count of taller-or-equal people in front of them. This is the most abstract lesson in the folder — sometimes the greedy insight isn't a formula, it's *which order to process things in* so the choices stop interfering with each other.

Complexity summary

#	Problem	Time	Extra space
01	lemonade_change	$O(n)$	$O(1)$
02	best_time_to_buy_and_sell_stock_ii	$O(n)$	$O(1)$
03	jump_game	$O(n)$	$O(1)$
04	jump_game_ii	$O(n)$	$O(1)$
05	gas_station	$O(n)$	$O(1)$
06	non_overlapping_intervals	$O(n \log n)$	$O(1)$ (excl. sort)
07	minimum_arrows_to_burst_balloons	$O(n \log n)$	$O(1)$ (excl. sort)
08	partition_labels	$O(n)$	$O(1)$ (bounded alphabet)
09	candy	$O(n)$	$O(n)$
10	queue_reconstruction_by_height	$O(n^2)$ (list insert is $O(n)$)	$O(n)$

10's $O(n^2)$ comes specifically from `list.insert` shifting elements — a linked-list-backed structure could bring it down to $O(n \log n)$ overall (still $O(n)$ sort dominates), worth mentioning as an optimization if asked.

Common interview follow-ups to be ready for

- `jump_game` / `jump_game_ii`: "can you return the actual jump sequence, not just whether it's possible / how many jumps?" — track which index each `curr_end` update came from, then reconstruct the path backward.
- `gas_station`: "how do you know the answer is unique if the total is non-negative?" — be ready to sketch the exchange-argument proof informally, not just assert it.
- `non_overlapping_intervals`: "why sort by end time instead of start time?" — sorting by start time doesn't work; be ready with a small counterexample showing why (a long interval starting first can block several short ones that would otherwise all fit).
- `minimum_arrows_to_burst_balloons`: "how is this different from `non_overlapping_intervals`?" — same sort, opposite framing: one counts rejections to **eliminate** overlap, the other counts selections to **cover** every interval; useful to state the relationship explicitly.
- `candy`: "can you do it in one pass instead of two?" — a trickier single-pass version exists (tracking ascending/descending run lengths), but the two-pass version here is the one to have solid first — mention the harder variant only if asked.
- `queue_reconstruction_by_height`: "why does tallest-first work but shortest-first doesn't?" — be ready to explain concretely why processing shorter people first breaks already-placed people's counts (a shorter person inserted before a taller one changes how many taller-or-equal people are in front of someone already placed).

Self-check before moving on

You should be able to, without looking at the code:

- State the exchange-argument shape in one sentence: "any optimal solution can be transformed into the greedy choice without making it worse."
- Explain why `jump_game_ii`'s "level" boundary (`curr_end`) update mirrors a BFS level, without prompting.
- Explain `gas_station`'s "skip the whole failed stretch" argument on a small hand-drawn example.
- State why interval-scheduling greedy problems sort by **end** time, with a concrete counterexample for why start-time sorting fails.
- Explain why `candy` needs two passes and why the second pass uses `max` instead of overwriting.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `16_intervals`.

Programs

01. Lemonade Change

01_lemonade_change.py

Explanation

Problem: Customers pay for a \$5 lemonade with a \$5, \$10, or \$20 bill, in the order given by `bills`. You start with no change. Return True if every customer can be given correct change.

Source : LeetCode 860 - Lemonade Change

Example:

Input: [5, 5, 5, 10, 20]

Output: True

Idea: the simplest possible greedy - at each step there's often only one *possible* choice, and where there's a real choice (making change for a \$20: either one \$10 + one \$5, or three \$5s), always prefer spending the *less flexible* bill first. A \$10 bill is only ever useful for making change for a \$20 - it can't help a future \$10 customer. A \$5 bill can make change for *either* a \$10 or a \$20. So when both options are available, spend the \$10 (keep the more versatile \$5s in reserve) - a greedy choice that's provably never worse than the alternative, since nothing is lost by keeping the more flexible resource around longer.

Code

```
def lemonade_change(bills: list[int]) -> bool:
    five = ten = 0

    for bill in bills:
        if bill == 5:
            five += 1
        elif bill == 10:
            if five == 0:
                return False
            five -= 1
            ten += 1
        else: # bill == 20
            if ten > 0 and five > 0:
                ten -= 1
                five -= 1
            elif five >= 3:
                five -= 3
            else:
                return False

    return True

if __name__ == "__main__":
    tests = [
```

```

        ([5, 5, 5, 10, 20], True),
        ([5, 5, 10, 10, 20], False),
        ([5, 5, 10], True),
        ([10, 10], False),
    ]

    for i, (bills, expected) in enumerate(tests, 1):
        got = lemonade_change(bills)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

02. Best Time To Buy And Sell Stock II

02_best_time_to_buy_and_sell_stock_ii.py

Explanation

Problem: Given daily stock prices, find the maximum profit achievable with as many buy/sell transactions as you like (must sell before buying again, no holding two positions at once).

Source : LeetCode 122 - Best Time to Buy and Sell Stock II

Example:

Input: [7, 1, 5, 3, 6, 4]

Output: 7 (buy at 1, sell at 5: +4; buy at 3, sell at 6: +3)

Idea: unlike [01_arrays_strings/07_best_time_to_buy_sell_stock](#) (single transaction only), unlimited transactions turn this into a greedy one-liner: capture *every* positive day-to-day price increase as its own tiny transaction. This is provably equivalent to the optimal answer - any longer buy-low-sell-high stretch decomposes exactly into the sum of its individual daily gains anyway (buying at the start and selling at the end of an upward run gives the same total as buying/selling at every up-day along the way), so there's no benefit to trying to identify longer runs explicitly.

Code

```

def max_profit(prices: list[int]) -> int:
    profit = 0
    for i in range(1, len(prices)):
        if prices[i] > prices[i - 1]:
            profit += prices[i] - prices[i - 1]
    return profit

if __name__ == "__main__":
    tests = [
        ([7, 1, 5, 3, 6, 4], 7),
        ([1, 2, 3, 4, 5], 4),
        ([7, 6, 4, 3, 1], 0),
        ([1], 0),
    ]

    for i, (prices, expected) in enumerate(tests, 1):
        got = max_profit(prices)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

03. Jump Game

03_jump_game.py

Explanation

Problem: Given an array where `nums[i]` is the maximum jump length from index `i`, determine if the last index is reachable starting from index 0.

Source : LeetCode 55 - Jump Game

Example:

Input: [2, 3, 1, 1, 4]

Output: True

Idea: track the single greedy quantity "farthest index reachable so far," updated at every position: `farthest = max(farthest, i + nums[i])`. There's no need to try every possible jump length from every position (which is what a backtracking/DP-over-every-choice approach would do) - only the *farthest* reach matters, since any jump length shorter than the max at a given index can never do better than the max jump already accounted for. If the current index `i` is ever beyond `farthest`, the walk has hit a gap no earlier jump could cross, so the end is unreachable.

Code

```
def can_jump(nums: list[int]) -> bool:
    farthest = 0
    for i, n in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + n)
    return True

if __name__ == "__main__":
    tests = [
        ([2, 3, 1, 1, 4], True),
        ([3, 2, 1, 0, 4], False),
        ([0], True),
        ([1, 0, 1], False),
    ]

    for i, (nums, expected) in enumerate(tests, 1):
        got = can_jump(nums)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

04. Jump Game II

04_jump_game_ii.py

Explanation

Problem: Same setup as 03 (`nums[i]` = max jump length from index `i`, starting at index 0, guaranteed reachable this time), but return the

minimum number of jumps needed to reach the last index.

Source : LeetCode 45 - Jump Game II

Example:

Input: [2, 3, 1, 1, 4]

Output: 2 (jump 0 -> 1 (index 0 to 1, len 1... actually 0->1 to index1, then jump 3 to index4): [2,3,1,1,4] -> jump to index 1, then jump 3 more to index 4)

Idea: an implicit BFS, expressed without an explicit queue - "one jump" covers an entire *range* of indices at once, like one level of a BFS frontier. **curr_end** marks the farthest index reachable with the jumps taken *so far*; **farthest** tracks the farthest reachable if one *more* jump were taken from anywhere within the current range. Walking through positions, the moment **i** reaches **curr_end** means every position in the current "level" has been considered - a jump must be taken to progress at all, so increment the jump count and set **curr_end = farthest** (moving to the next level/range). This mirrors [04_linked_list](#)'s level-by-level BFS shape, just over an implicit range-based graph instead of explicit nodes and edges.

Code

```
def jump(nums: list[int]) -> int:
    jumps = 0
    curr_end = 0
    farthest = 0

    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == curr_end:
            jumps += 1
            curr_end = farthest

    return jumps

if __name__ == "__main__":
    tests = [
        ([2, 3, 1, 1, 4], 2),
        ([2, 3, 0, 1, 4], 2),
        ([0], 0),
        ([1, 1, 1, 1], 3),
    ]

    for i, (nums, expected) in enumerate(tests, 1):
        got = jump(nums)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

05. Gas Station

05_gas_station.py

Explanation

Problem: n gas stations arranged in a circle; gas[i] is the fuel available at station i, cost[i] is the fuel needed to travel from station i to station i+1. Starting with an empty tank, return the starting station index that allows completing the full circuit, or -1 if none exists (the answer is guaranteed unique if it exists).

Source : LeetCode 134 - Gas Station

Example:

Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]

Output: 3

Idea: two greedy facts combine to solve this in one pass. First: a solution exists at all if and only if $\text{sum}(\text{gas}) \geq \text{sum}(\text{cost})$ overall - if the total fuel available can't cover the total cost, no starting point can possibly work, no matter where the trip begins. Second, and less obvious: if the tank goes negative starting from station *start* by the time it reaches station *i*, then *no* station between *start* and *i* (inclusive) could have been a valid starting point either - starting anywhere in that stretch only means arriving at each later station with *less* surplus than starting from *start* did, since the earlier stations' gas is what's forfeited. So the moment the running tank goes negative, jump the candidate start straight to *i + 1* and reset the tank to 0, skipping the need to separately retry every station in between.

Code

```
def can_complete_circuit(gas: list[int], cost: list[int]) -> int:
    total = 0
    tank = 0
    start = 0

    for i in range(len(gas)):
        diff = gas[i] - cost[i]
        total += diff
        tank += diff
        if tank < 0:
            start = i + 1
            tank = 0

    return start if total >= 0 else -1

if __name__ == "__main__":
    tests = [
        ([1, 2, 3, 4, 5], [3, 4, 5, 1, 2], 3),
        ([2, 3, 4], [3, 4, 3], -1),
        ([5, 1, 2, 3, 4], [4, 4, 1, 5, 1], 4),
    ]

    for i, (gas, cost, expected) in enumerate(tests, 1):
        got = can_complete_circuit(gas, cost)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

06. Non Overlapping Intervals

06_non_overlapping_intervals.py

Explanation

Problem: Given a collection of intervals, return the minimum number of intervals that must be removed so the rest don't overlap.

Source : LeetCode 435 - Non-overlapping Intervals

Example:

Input: [[1,2], [2,3], [3,4], [1,3]]

Output: 1 (remove [1,3])

Idea: the classic "activity selection" greedy - sort intervals by **end** time (not start time), then greedily keep an interval only if it starts at or after the previously kept interval's end. Sorting by end time is the crucial choice: keeping whichever interval ends **soonest** at every step leaves the most room for future intervals to also fit without overlapping - provably at least as good as any other selection strategy (an exchange argument: any valid selection can be modified to instead pick the earliest-ending option at each step without making things worse). Every interval that doesn't fit (starts before the previous kept interval ends) has to be one of the removed ones, so counting those rejections directly gives the answer - no need to explicitly construct the kept set.

Code

```
def erase_overlap_intervals(intervals: list[list[int]]) -> int:
    if not intervals:
        return 0

    intervals = sorted(intervals, key=lambda interval: interval[1])
    removed = 0
    prev_end = intervals[0][1]

    for start, end in intervals[1:]:
        if start < prev_end:
            removed += 1
        else:
            prev_end = end

    return removed

if __name__ == "__main__":
    tests = [
        ([[1, 2], [2, 3], [3, 4], [1, 3]], 1),
        ([[1, 2], [1, 2], [1, 2]], 2),
        ([[1, 2], [2, 3]], 0),
        ([], 0),
    ]

    for i, (intervals, expected) in enumerate(tests, 1):
        got = erase_overlap_intervals(intervals)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

07. Minimum Arrows To Burst Balloons

07_minimum_arrows_to_burst_balloons.py

Explanation

Problem: Balloons are represented as intervals $[x_start, x_end]$ along a line. An arrow shot at position x bursts every balloon whose interval contains x . Return the minimum number of arrows needed to burst all balloons.

Source : LeetCode 452 - Minimum Number of Arrows to Burst Balloons

Example:

Input: $[[10,16], [2,8], [1,6], [7,12]]$

Output: 2 (one arrow at $x=6$ bursts $[2,8]$ and $[1,6]$; one arrow at $x=12$ bursts $[10,16]$ and $[7,12]$)

Idea: the same end-time-sorted greedy shape as 06, applied to a question about *covering* points instead of *keeping* non-overlapping intervals - but they're really the same problem viewed from opposite sides. Sort by end coordinate; shoot the first arrow at the first balloon's end - the latest possible x that still bursts it, which therefore has the best chance of also bursting other balloons whose ranges extend that far. Any later balloon whose start is beyond the current arrow's position can't be hit by it, so it needs a new arrow, placed at *that* balloon's end going forward. Every balloon that starts before or at the current arrow's x is already covered - no separate tracking needed, since sorting by end time guarantees covered balloons are consumed in order.

Code

```
def find_min_arrow_shots(points: list[list[int]]) -> int:
    if not points:
        return 0

    points = sorted(points, key=lambda p: p[1])
    arrows = 1
    arrow_pos = points[0][1]

    for start, end in points[1:]:
        if start > arrow_pos:
            arrows += 1
            arrow_pos = end

    return arrows

if __name__ == "__main__":
    tests = [
        ([[10, 16], [2, 8], [1, 6], [7, 12]], 2),
        ([[1, 2], [3, 4], [5, 6], [7, 8]], 4),
        ([[1, 2], [2, 3], [3, 4], [4, 5]], 2),
        ([], 0),
    ]

    for i, (points, expected) in enumerate(tests, 1):
```

```

got = find_min_arrow_shots(points)
status = "PASS" if got == expected else "FAIL"
print(f"Test {i}: {status} (got {got}, expected {expected})")

```

08. Partition Labels

08_partition_labels.py

Explanation

Problem: Given a string *s*, partition it into as many contiguous pieces as possible such that each letter appears in at most one piece. Return the list of piece lengths.

Source : LeetCode 763 - Partition Labels

Example:

Input: "ababcbacadefegdehijhklij"

Output: [9, 7, 8]

Idea: precompute, for every character, the **last** index it occurs at in the whole string - that's the earliest a partition boundary could possibly be placed if this character has been seen in the current piece. Then scan left to right, extending the current piece's *end* to cover the last occurrence of every character encountered so far (same "extend to cover everything currently active" idea as [11_graphs/10_pacific_atlantic_water_flow](#)'s multi-source reasoning, just in 1D). The moment the scan position *i* reaches *end*, every character seen in this piece has had its last occurrence accounted for - none of them can appear again later, so it's safe to close the partition here and start a new one.

Code

```

def partition_labels(s: str) -> list[int]:
    last_occurrence = {ch: i for i, ch in enumerate(s)}
    result = []
    start = end = 0

    for i, ch in enumerate(s):
        end = max(end, last_occurrence[ch])
        if i == end:
            result.append(end - start + 1)
            start = i + 1

    return result

if __name__ == "__main__":
    tests = [
        ("ababcbacadefegdehijhklij", [9, 7, 8]),
        ("eccbbbbbdec", [10]),
        ("abc", [1, 1, 1]),
    ]

    for i, (s, expected) in enumerate(tests, 1):
        got = partition_labels(s)
        status = "PASS" if got == expected else "FAIL"

```

```
print(f"Test {i}: {status} (got {got}, expected {expected})")
```

09. Candy

09_candy.py

Explanation

Problem: Children stand in a line, each with a rating. Every child gets at least one candy, and any child with a higher rating than an immediate neighbor must get more candy than that neighbor. Return the minimum total candies needed.

Source : LeetCode 135 - Candy

Example:

Input: [1, 0, 2]

Output: 5 (candies [2, 1, 2])

Idea: the constraint is **bidirectional** (compare to both the left neighbor and the right neighbor), but each single greedy left-to-right or right-to-left pass can only correctly enforce the comparison in **one** direction at a time. The fix: two passes. A left-to-right pass enforces "higher than the left neighbor gets more candy than the left neighbor" by only ever increasing from the previous position. A right-to-left pass then enforces the mirror constraint against the right neighbor - using `max(candies[i], candies[i+1] + 1)`, not a plain overwrite, since the left-to-right pass may have already assigned this position a value satisfying its **left** constraint that's bigger than what the right constraint alone would require, and that already-satisfied value must not be reduced.

Code

```
def candy(ratings: list[int]) -> int:
    n = len(ratings)
    candies = [1] * n

    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            candies[i] = candies[i - 1] + 1

    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            candies[i] = max(candies[i], candies[i + 1] + 1)

    return sum(candies)

if __name__ == "__main__":
    tests = [
        ([1, 0, 2], 5),
        ([1, 2, 2], 4),
        ([1, 3, 2, 2, 1], 7),
        ([1], 1),
    ]
```

```

for i, (ratings, expected) in enumerate(tests, 1):
    got = candy(ratings)
    status = "PASS" if got == expected else "FAIL"
    print(f"Test {i}: {status} (got {got}, expected {expected})")

```

10. Queue Reconstruction By Height

10_queue_reconstruction_by_height.py

Explanation

Problem: Given people as pairs [h, k] - height h, and k = the number of people in front of this person with height $\geq h$ - reconstruct and return the queue (any order satisfying every pair's constraint).

Source : LeetCode 406 - Queue Reconstruction by Height

Example:

Input: [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]

Output: [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]

Idea: the trickiest greedy in this folder because the two coordinates pull in different directions and have to be untangled in the right order. Sort by height **descending**, and for equal heights, by k **ascending**. Then insert people one at a time, each at index k in the result built so far. Why this works: processing tallest-first means every person already placed is $\geq$ the current person's height, so inserting the current person at position k automatically has exactly k taller-or-equal people in front of them - insertion doesn't disturb that count for people already placed, either, since inserting a **shorter** person anywhere never counts toward any already-placed taller person's own k requirement. This is the general lesson from this greedy: when two constraints interact, look for an order to process them in where satisfying one **doesn't retroactively break** the other - here, tallest-first is exactly that order.

Code

```

def reconstruct_queue(people: list[list[int]]) -> list[list[int]]:
    people = sorted(people, key=lambda p: (-p[0], p[1]))
    result: list[list[int]] = []
    for person in people:
        result.insert(person[1], person)
    return result

if __name__ == "__main__":
    tests = [
        ([7, 0], [4, 4], [7, 1], [5, 0], [6, 1], [5, 2]),
        ([5, 0], [7, 0], [5, 2], [6, 1], [4, 4], [7, 1]),
        ([6, 0], [5, 0], [4, 0], [3, 2], [2, 2], [1, 4]),
        ([4, 0], [5, 0], [2, 2], [3, 2], [1, 4], [6, 0]),
    ]

    for i, (people, expected) in enumerate(tests, 1):
        got = reconstruct_queue(people)
        status = "PASS" if got == expected else "FAIL"

```

```
print(f"Test {i}: {status} (got {got}, expected {expected})")
```